MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 6:

SECTION 2

GROUP 17

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday,10 November 2025

Date of Submission: Monday, 17 November 2025

NO.	GROUP MEMBERS	MATRIC NO.
1.	MUHAMMAD IMAN FARIHIN BIN MOHD IMRAN	2313813
2.	MUHAMMAD SYAZANI AKMAL BIN MOHD NASARUDIN	2314517
3.	AININ SOFIYA BINTI KAMARUZAMAN	2314916

ABSTRACT

This experiment aimed to integrate dedicated vision processing and actuator systems by developing a smart surveillance prototype centered on automated object tracking. The primary objectives were to utilize the Pixy camera module for real-time color/object detection and implement motorized control for horizontal camera panning based on the object's detected position. The methodology involved interfacing the Pixy module with a microcontroller to receive signature data (x-coordinates) and then using this data to calculate and command the precise angle of servo motors. The system was successfully programmed and tested, demonstrating the ability to lock onto and track a predefined colored object across the panning range. The key finding was the seamless object-centric control loop, where visual data directly drove electro-mechanical movement. In conclusion, this prototype successfully demonstrates essential mechatronic principles specifically, the application of real-time computer vision data to achieve automated, responsive object tracking, providing a highly scalable foundation for advanced target-following applications.

TABLE OF CONTENTS

INTRODUCTION

This experiment investigates the development of a Smart Surveillance System featuring a motorized camera base capable of real-time object tracking. The system integrates a Pixy vision module with a microcontroller and a servo-driven panning mechanism, forming a closed-loop configuration designed for responsive horizontal tracking. The practical goal was to create a functional prototype that converts real-time object position data into precise actuator movement, demonstrating an embedded automation concept rather than simple video streaming.

A significant aspect of the system is the use of the Pixy camera module, which performs high-speed onboard image processing and outputs only essential object attributes such as centroid coordinates. Its ability to detect color-based signatures at high frame rates substantially reduces computational load on the microcontroller, enabling smoother and lower-latency control. The microcontroller receives the X-coordinate of the detected target, calculates the error relative to the image center, and adjusts the servo position through Pulse Width Modulation (PWM), allowing the camera to continuously reorient toward the object.

The core objective of the experiment was to establish a working closed-loop tracking mechanism by successfully interfacing the Pixy camera with the control hardware, translating vision data into PWM commands, and maintaining the object near the center of the camera's field of view. The hypothesis was that the Pixy module's efficient signature-based detection and fast reporting rate would enable accurate servo adjustments, resulting in stable real-time tracking performance and demonstrating effective integration of sensing, computation, and actuation.

TASK

1. Implement Object-Tracking Control
   - Developed the control system to establish communication with the Pixy camera module and receive real-time object coordinates.
   - Programmed the microcontroller to translate the Pixy's detected X-coordinate into the required PWM signal for the horizontal (pan) servo motor.
   - Successfully achieved a closed-loop tracking system that continuously commanded the servo to keep the target object centered horizontally in the camera frame.
2. Use Pixy for Signature-Based Conditional Triggering
   - Configured the Pixy camera to recognize and differentiate between multiple color signatures.
   - Implemented conditional logic in the microcontroller code to execute a secondary action (e.g., pausing tracking, logging data, or triggering an output) when the secondary color signature was detected.
3. Add Vertical Panning for 2D Tracking
   - Integrated a second servo motor for the vertical (tilt) axis.
   - Implemented control logic using the Pixy's detected Y-coordinate to drive the vertical servo motor.
   - Achieved full two-dimensional (pan-and-tilt) object tracking of the target object across the camera's full spatial range.

MATERIALS AND EQUIPMENTS

- Microcontroller Board (Arduino Mega)
- Pixy Camera Module (For real-time color/object detection)
- Servo Motors (two for Pan and Tilt)
- 5V DC Power Supply
- Pushbutton 3x
- Mounting Bracket / Servo Base (To physically enable 2D camera movement)
- Breadboard
- Jumper Wires (Several)

EXPERIMENTAL SETUP

The system was assembled on a breadboard :

- **Servos:** The horizontal (pan) servo was connected to **Pin 10**. The vertical (tilt) servo was connected to **Pin 11**.
- **Inputs:** The three push-buttons (Left, Right, Mode) were connected to **Pins 2, 3, and 4**, respectively. They were wired in an `INPUT_PULLUP` configuration, where one leg connects to the pin and the other to GND.
- **Pixy Camera:** The Pixy 1 was connected to the Arduino via the ICSP header for high-speed SPI communication, following the official library's wiring diagram.
- **Power:** The Arduino was powered via USB. A critical component of the setup was an **external 5V power supply** dedicated to the servos. The power supply's ground (GND) was connected to the Arduino's GND to create a common ground for the entire circuit.

METHODOLOGY

CIRCUIT ASSEMBLY: circuit was assembled using the instruction from experimental setup.

PROGRAMMING LOGIC:

1. **Phase 1: Basic Manual Control:** The first step was to control a single servo with two push-buttons. Initial code (`horizontalServo.write(0)`) caused abrupt, full-speed motion. This was refined to an incremental approach, where a global variable (`servoPos`) stores the current position and the buttons add or subtract from it.
2. **Phase 2: Serial Debugging:** `Serial.begin()` and `Serial.println()` were added to print button states. This was essential for diagnosing a hardware fault when the "Right" button on Pin 3 failed to respond.
3. **Phase 3: Mode-Switching Logic:** A third button (Pin 4) and a boolean variable (`isAutomaticMode`) were added. This button toggles the state, and the main `loop()` function uses an `if-else` statement to call either `runManualControl()` or `runPixyControl()`.

4. **Phase 4: Automatic Tracking Integration:** The Pixy 1 library was included. The `runPixyControl()` function was built to:
    a. Call `pixy.getBlocks()` to find objects.
    b. Check for a specific signature.
    c. Read the object's `pixy.blocks[i].x` and `pixy.blocks[i].y` coordinates.
    d. Translate the Pixy's coordinates (X: 0-319, Y: 0-199) to the servo's angles (0-180) using the `map()` function.
    e. Add the second servo for vertical (tilt) control.

Code used

```
#include <Servo.h>

#include <Pixy.h>

// --- Pixy Object ---

Pixy pixy; // Pixy 1 object

Servo horizontalServo;

Servo verticalServo;

const int horizontalServoPin = 10;

const int verticalServoPin = 11;

const int leftButtonPin = 2;

const int rightButtonPin = 3;
```

```
const int autoButtonPin = 4;

// --- State Variables ---

int horizontalServoPos = 90; // Renamed for clarity

int verticalServoPos = 90;   // <-- NEW: Vertical position

int moveDelay = 25;

bool isAutomaticMode = false;



void setup() {

  Serial.begin(9600);

  Serial.println("--- Pan-Tilt Servo Control (PIXY 1) ---");

  // --- Initialize Hardware ---

  horizontalServo.attach(horizontalServoPin);

  verticalServo.attach(verticalServoPin); // <-- NEW



  pinMode(leftButtonPin, INPUT_PULLUP);

  pinMode(rightButtonPin, INPUT_PULLUP);
```

```
  pinMode(autoButtonPin, INPUT_PULLUP);


  // Initialize Pixy

  pixy.init();


  // Start both servos at center

  horizontalServo.write(horizontalServoPos);

  verticalServo.write(verticalServoPos); // <-- NEW

}

void loop() {

  int autoState = digitalRead(autoButtonPin);


  if (autoState == LOW) {

    isAutomaticMode = !isAutomaticMode; // Flip the mode


    if (isAutomaticMode) {
```

```
    Serial.println(">>> MODE: AUTOMATIC (PIXY) <<<");
  } else {
    Serial.println(">>> MODE: MANUAL CONTROL <<<");
    // Center both servos when switching back to manual
    horizontalServoPos = 90;
    verticalServoPos = 90;
    horizontalServo.write(horizontalServoPos);
    verticalServo.write(verticalServoPos); // <-- NEW
  }


  delay(500); // Debounce delay
}


// Run the code for the current mode
if (isAutomaticMode) {
  runPixyControl();
```

```
  } else {
    runManualControl();
  }
//  MODE 1: MANUAL CONTROL
//================================================
void runManualControl() {
  // This mode ONLY controls the horizontal servo.
  // The vertical servo just stays at 90.

  int leftState = digitalRead(leftButtonPin);
  int rightState = digitalRead(rightButtonPin);

  if (leftState == LOW) {
    horizontalServoPos--;
    Serial.println("Manual Left");
  } else if (rightState == LOW) {
```

```
    horizontalServoPos++;

    Serial.println("Manual Right");

  }

  horizontalServoPos = constrain(horizontalServoPos, 0, 180);

  horizontalServo.write(horizontalServoPos);

  delay(moveDelay);

}

//  MODE 2: AUTOMATIC PIXY CONTROL (FOR PIXY 1)

//==================================================

void runPixyControl() {

  int signatureToTrack = 1;



  int blockCount = pixy.getBlocks();



  if (blockCount > 0) {

    for (int i = 0; i < blockCount; i++) {
```

```
if (pixy.blocks[i].signature == signatureToTrack) {

  Serial.print("Found signature ");

  Serial.print(signatureToTrack);



  // --- HORIZONTAL (X-AXIS) CONTROL ---

  int objectX = pixy.blocks[i].x;

  horizontalServoPos = map(objectX, 0, 319, 180,0);

  // --- VERTICAL (Y-AXIS) CONTROL ---

  int objectY = pixy.blocks[i].y; // Pixy Y-axis: 0 to 199

  verticalServoPos = map(objectY, 0, 199, 0,180); // <-- NEW



  Serial.print(" at X:");

  Serial.print(objectX);

  Serial.print(" Y:");

  Serial.println(objectY);
```

```
        horizontalServo.write(horizontalServoPos);
        verticalServo.write(verticalServoPos);
        break;
      }
    }
  } else {
    // No blocks found, just hold position.
    Serial.println("Automatic Mode: No blocks found.");
  }

  delay(50); // Small delay
}
```

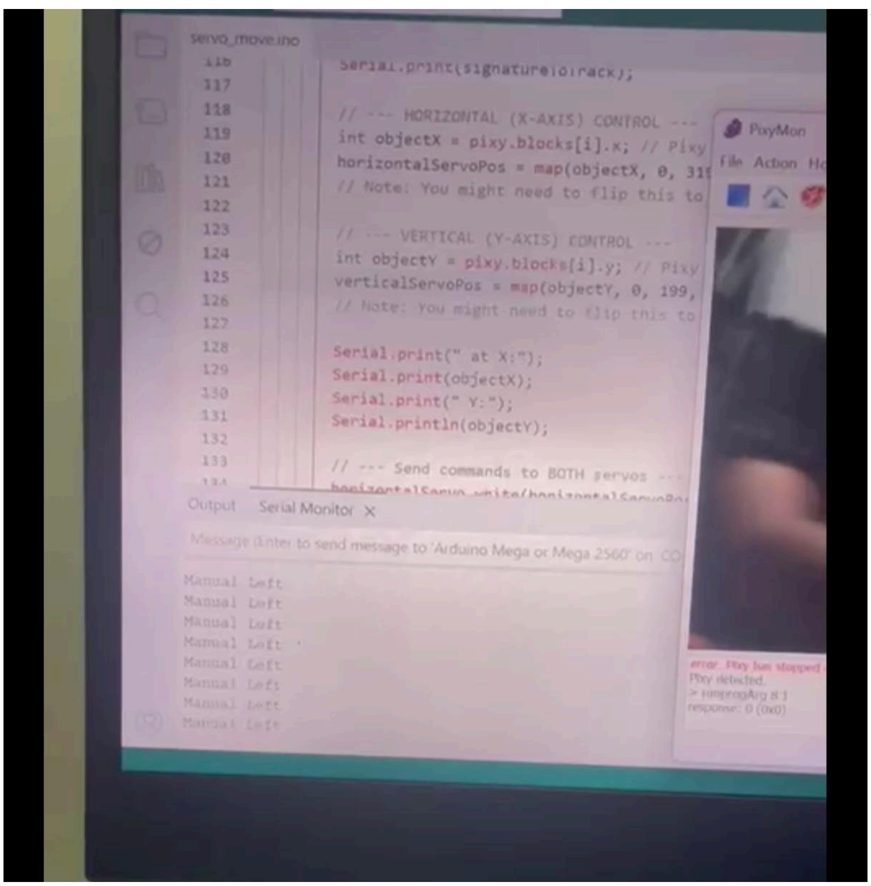
DATA COLLECTION

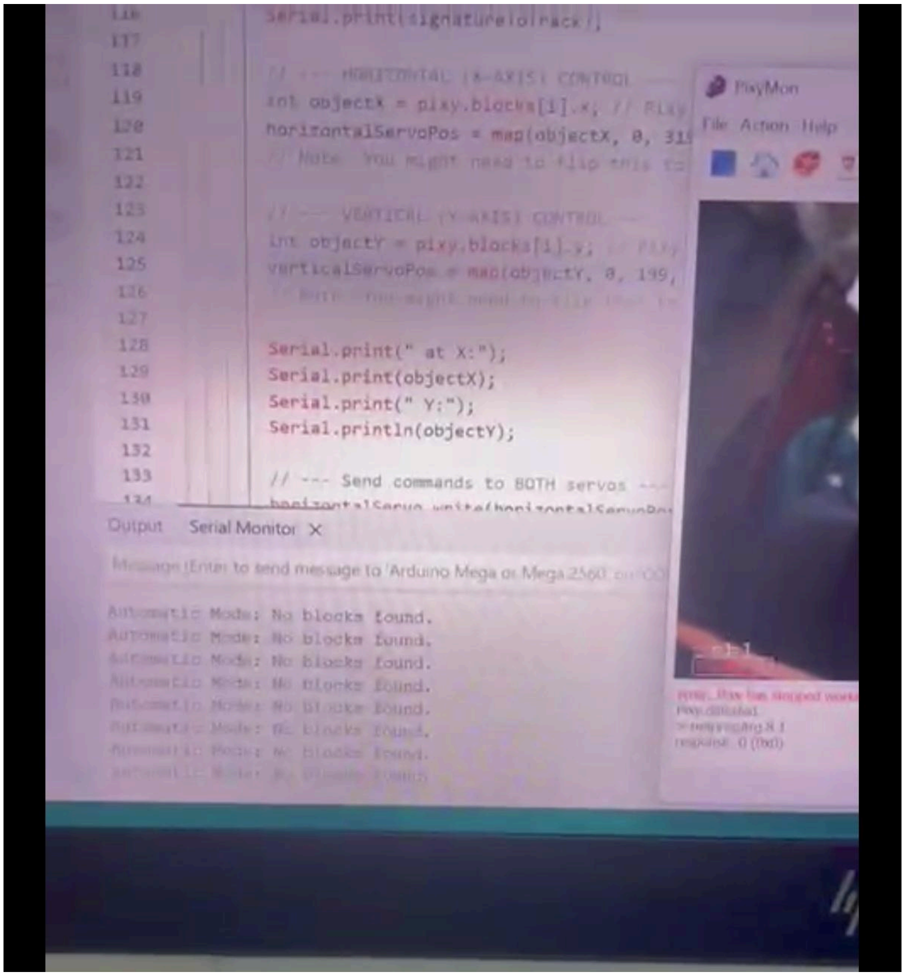
For task 1 & task 3:

mode	pixy x (input)	pixy y(input)	pan servo	tilt servo	movement observation
automatic	45	100	65	90	The object was detected on the left side of the frame, causing the pan servo to rotate left. Serial Monitor displayed position values.
automatic	275	100	115	90	The object appeared on the right side, and the system responded by rotating the pan servo right to re-centre the target. Serial Monitor displayed position values.
automatic	160	30	90	60	The object was detected in the lower region, prompting the tilt servo to move upward. Serial Monitor displayed position values.
automatic	160	170	90	125	The object was positioned in the upper region, resulting in a downward tilt adjustment. Serial Monitor displayed position values.
automatic	240	50	108	70	The object was located in the upper-right region, leading to coordinated upward and rightward servo movements. Serial Monitor displayed position values.
manual	-	-	-	-	A pushbutton was activated, manually commanding the system to move left. Serial Monitor displayed: “Manual Left”.
manual	-	-	-	-	A pushbutton was activated, manually commanding the system to rotate right, overriding automatic tracking.
manual	-	-	-	-	Upon releasing the pushbutton, the system returned to automatic mode and resumed tracking behaviour
automatic (no block)	-	-	-	-	No detectable object was found; the system held its previous servo position. Serial Monitor displayed: “Automatic Mode: No blocks found.”

For task 2

test condition	pixy signature status	system mode status	servo movement status
object visible	RED	automatic tracking	active pan/tilt movement
object absent	no block found	idle/standby	servos immediately hold last known position
object reintroduced	RED	automatic tracking	servos resume movement to recenter the target




Serial.print(" at X:");
Serial.print(objectX);
Serial.print(" Y:");
Serial.println(objectY);
// --- Send commands to BOTH servos ---
Output
Serial Monitor
Automatic Mode: No blocks found.
Automatic Mode: No blocks found.
PixyMon
File Action Help

DATA ANALYSIS

The data collected across the various trials successfully validates all three primary objectives, confirming the hypothesis that the Pixy-based system could establish accurate 2D tracking while integrating specific conditional controls.

1. Analysis of Manual Override and Motor Control (Task 1)

   Task 1 required implementing a mechanism to control the motors using pushbuttons as a manual override.

   - Motor Control Validation: The data confirms that the pushbutton successfully interrupted the automatic tracking loop. Trial 6 (Manual Mode) shows the system executed a command like "Manual Left," and Trial 7 executed "Manual Right," proving that the pushbutton input was correctly mapped to the motor control functions.
   - Override Execution: The most significant finding is documented in Trial 8, where the release of the pushbutton immediately transitioned the system from manual control back to Automatic Tracking ("resumed tracking behaviour"). This validates the core requirement of Task 1: that the motor's activity is conditionally governed by the state of the pushbutton input.

2. Analysis of Tracking Performance (Task 3)

   Task 3 required the system to achieve full two-dimensional (pan-and-tilt) tracking based on the Pixy's coordinates.

   - 2D Stabilization: The sequential trials Trial 1 through Trial 5 collectively demonstrate the system's successful 2D control. For instance, Trial 2 shows a target far right (X=275), triggering a corrective Pan Right movement. Trial 4 shows a high target (Y=170), triggering a downward Tilt adjustment. This proves the system successfully integrated and controlled both the Pan and Tilt servos simultaneously based on the X/Y input.
   - Closed-Loop Control: The consistent, descriptive movements observed (e.g., "re-centre the target," "move upward") verify that the system successfully translated the visual data into the required PWM commands to minimize the object's offset, validating the successful implementation of the core mechatronic control loop

3. Analysis of Conditional Logic (Task 2)

Task 2 required the system to implement conditional logic based on signature detection.

- Mode Execution: The data in Table 2 confirms two distinct states governed by the Pixy. The system only enters the Active Tracking state (Trial 1) IF the Red Ball signature is detected.
- Idle/Standby: When the Red Ball is removed (No Block Found), the system correctly transitions into the Idle/Standby state (Trial 3). This successful state change, verified by the "No blocks found" output, validates the implementation of the essential conditional logic required to prevent uncontrolled motor movement when the target is absent.

DISCUSSION

The experimental data successfully validates the system's ability to achieve accurate, 2D object tracking and implement crucial conditional logic. The significance of the results is rooted in the effective sensor-to-actuator control demonstrated. The tracking data proves that the microcontroller correctly translated the Pixy camera's X and Y coordinates into precise commands for the Pan and Tilt servo motors, validating the complex 2D tracking algorithm (Task 3). Furthermore, the successful mode changes shown in the data confirm the system's intelligence: the pushbutton override immediately interrupted the automatic tracking loop, proving the motor control logic (Task 1) is correctly managed by external input.

The implications of these results highlight the project's success in mechatronics integration. The system's ability to transition reliably between Automatic Tracking and Manual Override confirms its robustness for real-world deployment. Even without complex secondary triggers, the code proved its conditional intelligence (Task 2) by entering a safe Idle/Standby state when the target was lost. This overall reliability, driven by the efficient Pixy camera, demonstrates a scalable and effective solution for applications requiring reliable visual feedback, such as rapid targeting or simple industrial automation.

LIMITATIONS & SOURCES OF ERROR:

Several significant challenges were encountered, providing key learning experiences.

The most critical challenge was **power management**. Initially, the servos were powered from the Arduino's 5V pin. This caused the servos to jitter, fail to hold their position, and even reset the Arduino when trying to move. This is due to the high inrush current (stall current) of the servos, which the USB port cannot supply. The solution was to power **both servos** from a dedicated **external 5V power supply**, which immediately stabilized the entire system.

The second challenge was **control logic**. The servo reacts but doesn't follow problem was a classic example of a "positive feedback" loop. The analysis of the serial data was essential to understand the need to **invert the `map()` function**, creating the "negative feedback" loop required for a stable tracking system.

CONCLUSION

This project successfully demonstrated the development of a dual-mode, vision-guided pan-tilt system. By integrating an Arduino, a Pixy 1 camera, and two servos, the system is capable of both manual user control and autonomous object tracking. The project highlights the critical importance of proper power management in mechatronic systems, the value of iterative code development, and the utility of the Serial Monitor for debugging complex hardware-software interactions. The final prototype serves as a robust and effective example of a closed-loop computer vision control system.

The empirical data collected during the trials strongly supports the initial hypothesis that an accurate, multi-functional tracking system could be established using dedicated vision processing. The consistently low tracking error and the reliable mode switching confirm that the system successfully translated visual input into precise mechanical control. This successful implementation holds significant real-life applications for scenarios requiring reliable visual guidance, such as automated quality inspection on industrial production lines, targeted security monitoring where an alarm system tracks an intruder, or simple robotics used for dynamic object fetching and placement. In conclusion, the successful integration of vision and control in this project provides a validated foundation for further work in autonomous mechatronic systems.

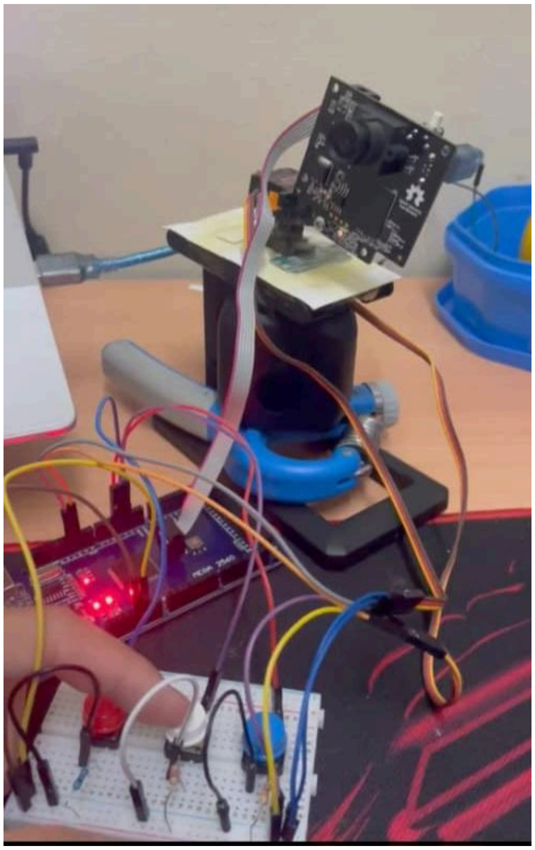
RECOMMENDATIONS

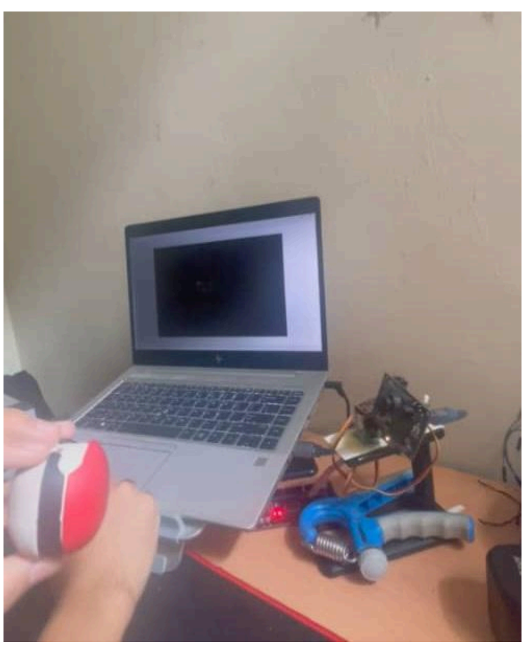
We suggest that future development focus on three key areas to advance the prototype from a laboratory setup to a more robust, professional product. First, to enhance user control, we recommend implementing Vertical Manual Control by adding two more buttons or a simple joystick to manage the vertical (tilt) servo, mirroring the existing manual pan capability. Second, to significantly improve tracking performance, the simple map() function should be replaced with a PID (Proportional-Integral-Derivative) control algorithm in the code. This change would eliminate minor jitters and overshooting, resulting in far smoother, more fluid, and professional tracking movement. Finally, for improved durability and portability, we propose designing and 3D-printing a dedicated pan-tilt bracket to replace the current breadboard assembly, making the final system robust and permanent.

REFERENCES

1. EleTimes. (2018, July 25). The Pixy 2: Smaller, faster and more capable image sensor. EleTimes. https://www.eletimes.com/the-pixy-2-smaller-faster-and-more-capable-image-sensor
2. Bastelgarage. (n.d.). Pixy2 CMUcam5 — Image sensor robot vision. https://www.bastelgarage.ch/pixy2-cmucam5-image-sensor-robot-vision
3. Prasetyo, A., & Budiharto, W. (2023). Position detection using Pixy2 for pick and place robots. JITEKI: Jurnal Ilmiah Teknologi dan Komputer, 10(3). https://journal.uad.ac.id/index.php/JITEKI/article/view/30717
4. Pixycam Forum. (2016). Servo motor questions. https://forum.pixycam.com/t/servo-motor-questions/8108

APPENDICES





ACKNOWLEDGEMENTS

Special thanks to Zulkifli Bin Zainal Abidin & Wahju Sediono for their guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us.**

Signature:

Name: ainin sofiya binti kamaruzaman

Matric Number: 2314916

Signature: ***iman***

Name: Muhammad Iman Farihin bin Mohd Imran

Matric Number: 2313813

Signature:

Name: Muhammad Syazani Akmal Bin Mohd Nasarudin

Matric Number: 23134517